BSCS FINAL PROJECT
Design and Test Specification

ReMIND: Generative AI Based Memory Rebuilder

Signed By Zulkifl

Project Advisor

Muhammad Zulkifl Hassan

Presented by:
Group ID: F25CS104

Registration No:	**Name:**
L1F22BSCS0086	M. Hassan Ashraf
L1F22BSCS0103	M. Talha Faizan
L1F22BSCS0094	Mubashar Tanveer

Faculty of Information Technology & Computer Science

University of Central Punjab

Design and Test Specification

DTS Phase II

ReMIND: Generative AI based Memory Rebuilder

Advisor: Muhammad Zulkifl Hasan

Team F25CS0104

Member Name	Primary Responsibility
M. Hassan Ashraf	AI, Research and Data Analysis
M. Talha Faizan	Frontend and APIs
Mubashar Tanveer	Backend and Testing

Table of Contents

Revision History

Name	Date	Reason For Changes	Version

Previous Phases Feedback

Idea Defence Feedback (Screenshot)

F25 Idea Defence Feedback

PO Project Office

Reply Reply all Forward

To: L1F22BSCS0086- MUHAMMAD HASSAN ASHRAF; L1F22BSCS0103- MUHAMMAD TALHA FAIZAN; **+1 other** Mon 01/09/2025 10:35

Cc: Dr Adnan Ghafoor; Muhammad Zulkifl Hasan

Dear Students,

Following is the feedback regarding your Idea Defence:

Sr. #	**CS-15**
Project Title	**GENERATIVE AI BASED MEMORY REBUILDER**

Similar products already exist. If the patient/a family member also does not have the details of event which we want to restore. Then system cannot perform.

Problem statement not clear. Scope not clear.

Students are incapable of implementing the idea. They don't know the basics of the project. Better to change the idea.

Major revision required. Revise your project idea in light of comments provided. Last date to submit the revised idea form is **Thursday, September 11, 2025, before 3:00 PM** in Project Office (F-304 Building A).

Phase 2 (SDS) Feedback (Screenshot)

PO

Project Office

To: L1F22BSCS0086- MUHAMMAD HASSAN ASHRAF; L1F22BSCS0103- MUHAMMAD TALHA FAIZAN; L1F22BSCS0094- MUBASHAR TANVEER

Thu 30/04/2026 13:38

Cc: Muhammad Zulkifl Hasan

Dear Students,

Following is the feedback regarding your phase 2 (SDS):

F25CS104	**REMIND: GENERATIVE AI BASED MEMORY REBUILDER**

Redraw class diagram. Should be online deployed. Must compare with at-least one available app.

Workflow knowledge was missing.

Abstract

ReMIND is an AI assisted web application designed to support individuals experiencing memory loss disorders such as Alzheimer's disease, dementia, and age-related cognitive decline. Memory loss affects not only a person's ability to recall past events but also their emotional stability, relationships, and overall sense of identity. Most existing digital solutions rely heavily on manual input, and when no prior record or recollection exists, they fail to assist patients and caregivers in restoring meaningful experiences. ReMIND addresses this limitation by offering semi-automated memory reconstruction that helps users reconnect with their personal history.

The system collects and analyzes different forms of data, including written notes, photographs, and voice recordings, and organizes them into personalized timelines that reflect moments from daily life. Through artificial intelligence, ReMIND generates contextual details that can suggest likely places, people, and events related to the user's memories. This enriched information helps patients retrieve forgotten experiences in a more natural and emotionally resonant way. The platform creates an interactive environment where patients and caregivers can revisit, discuss, and refine reconstructed memories together, strengthening emotional connection and supporting cognitive engagement.

The project brings together concepts from artificial intelligence, natural language processing, computer vision, speech processing, data management, and human centered design. The focus is not solely on automation but on creating an experience that prioritizes empathy, accessibility, and user comfort. By transforming fragmented or incomplete data into meaningful memory narratives, ReMIND aims to restore a sense of identity and continuity for individuals facing cognitive decline.

The expected outcome is a functional prototype that demonstrates how intelligently guided memory reconstruction can contribute to cognitive rehabilitation and long-term digital memory preservation. ReMIND highlights the potential of emerging AI systems to support emotional wellbeing and provide hope for individuals whose memories are fading.

1. Introduction

ReMIND is an AI-assisted web application aimed at helping individuals with memory loss, such as Alzheimer's and dementia patients, relive past experiences. It reconstructs fragmented memories using multimedia inputs. The project is motivated by the emotional and cognitive challenges faced by patients, emphasizing empathy, recall, and caregiver support through technology.

1.1 Product

ReMIND is an intelligent, web-based memory reconstruction system designed to assist individuals suffering from memory loss disorders such as Alzheimer's and dementia. The software addresses the limitations of existing reminder tools that rely heavily on manual event logging or user recall, which often fail when sufficient details are unavailable. ReMIND integrates multiple data sources such as photos, text notes, and voice recordings to automatically identify, organize, and recreate meaningful events. Using artificial intelligence and contextual analysis, the system reconstructs daily timelines and generates narrative memories that patients and caregivers can revisit. Its user-friendly, elderly-oriented interface ensures accessibility and comfort, while secure backend services manage data storage and processing. The system provides collaborative features for caregivers, enabling them to contribute and review reconstructed memories. Ultimately, ReMIND bridges emotional and cognitive gaps, improving quality of life by transforming fragmented inputs into coherent, emotionally engaging experiences that support recall and human connection.

1.2 Background

The project lies within the domains of artificial intelligence, cognitive rehabilitation, and healthcare technology. It focuses on applying AI techniques such as natural language processing, computer vision, and speech processing to assist individuals with memory loss. By merging technology and empathy, ReMIND enhances memory recall, emotional well-being, and caregiver collaboration.

1.3 Objective(s)/Aim(s)/Target(s)

The primary objective of ReMIND is to develop an AI-assisted web application that helps individuals with memory loss to reconstruct meaningful memories from fragmented multimedia inputs. The project aims to combine technology and empathy to create a system that supports both patients and caregivers in recalling and preserving life experiences.

The specific objectives are as follows:

Memory Reconstruction: Transform incomplete data such as text notes, photos, and voice recordings into coherent and contextually rich memory narratives.

Event Clustering: Use AI techniques to organize related media and information into structured timelines representing daily or significant events.

Reminder and Notification Module: Provide timely reminders and alerts to assist patients in maintaining routine and connection with daily life.

Elderly-Friendly Interface: Design a simple, accessible interface optimized for older users and non-technical caregivers.

Secure Data Handling: Implement encryption, authentication, and cloud storage to protect sensitive personal data.

Caregiver Collaboration: Enable caregivers to contribute and review reconstructed memories to ensure accuracy and emotional value.

Proof of Concept Demonstration: Deliver a functional prototype using existing AI APIs and web technologies to validate the concept.

Overall, ReMIND seeks to enhance emotional well-being, cognitive support, and human connection through accessible AI innovation.

1.4 Scope

The scope of ReMIND covers the design and development of a web-based application that reconstructs and presents memories using multimedia inputs such as text, images, and audio. The system includes event clustering, memory timeline generation, and a reminder module for patients and caregivers. It focuses on enhancing recall and emotional connection through AI-powered contextual analysis. The application will feature an elderly-friendly interface, secure data handling, and cloud-based deployment. Medical diagnosis, hospital EHR integration, and real-time biometric tracking are excluded from the project. It will be a Concept-of-Proof limited project.

1.5 Business Goals

As a proof-of-concept prototype, ReMIND aims to demonstrate the commercial potential of AI-driven memory reconstruction in healthcare and assisted living. Its primary goal is to validate the feasibility of using artificial intelligence for emotional and cognitive support in patients with memory loss. The project establishes a foundation for future development into a scalable product that could benefit healthcare institutions, caregivers, and families. By integrating accessible technology with empathetic design, ReMIND highlights opportunities for collaboration between AI developers and healthcare providers, showcasing how innovation can lead to meaningful, socially impactful digital health solutions in memory rehabilitation.

1.6 Document Conventions

The Document Conventions being used throughout the document are following:

- Text being used is Times New Roman.
- Main Headings are of Size 16.
- Sub-Headings are of Size 14.
- Content/Paragraph and Tertiary headings are of Size 12.
- Auto line spacing is used.

2. Technical Architecture

This section describes the overall technical architecture of the ReMIND system, including major subsystems, processing responsibilities, interfaces, data components, technology stack, and hosting considerations. The architecture incorporates both the operational prototype and the envisioned full-system interactions.

ReMIND is a **custom-built application that leverages Commercial Off-The-Shelf (COTS) AI services**, primarily inspired by and built around the **ChatGPT-style generative AI architecture**. Rather than developing AI models from scratch, the system integrates mature third-party AI APIs for language understanding, image enhancement, speech transcription, and voice synthesis. This approach ensures technical feasibility within constraints while maintaining functional depth.

At a high level, ReMIND follows a **layered, client–server architecture** that supports **online, near real-time processing**. The system handles event-driven transactions such as multimedia uploads, AI processing requests, memory reconstruction, and reminder generation. Analytical reporting is limited to basic logs and feedback analysis, consistent with the proof-of-concept nature of the project.

Major Application Components and Responsibilities:

The **front-end layer** provides a browser-based user interface optimized for elderly users. It enables multimedia input, reconstructed memory visualization, reminder interaction, and feedback submission.

The **backend layer** acts as the core control unit. It manages authentication, workflow orchestration, API communication, and business logic. It coordinates AI requests, validates outputs, and ensures secure storage and retrieval of data.

The **AI orchestration module** serves as a mediator between the backend and external AI services. It aggregates multimodal outputs and initiates memory reconstruction via large language models.

The **data layer** manages structured and unstructured data, including user profiles, memory timelines, reminders, and media references.

2.1 Application and Data Architecture

Component Diagram:

This Component Diagram illustrates the physical architecture of the ReMiND system, showing how the software components interact. The core communication hub is the API Gateway, which routes requests between different modules. The Frontend (React.js) provides user interfaces like the Memory Input UI and Reconstructed Memories Viewer. The Backend (Flask / FastAPI) handles User Management and Event Clustering. The system relies on external AI Services for tasks like Story Generation and Speech-to-Text. Data is persistently stored in a PostgreSQL Database and Cloudinary / Firebase Media Storage. The Notification Module handles reminders, and the entire system is deployed using Render and Heroku.

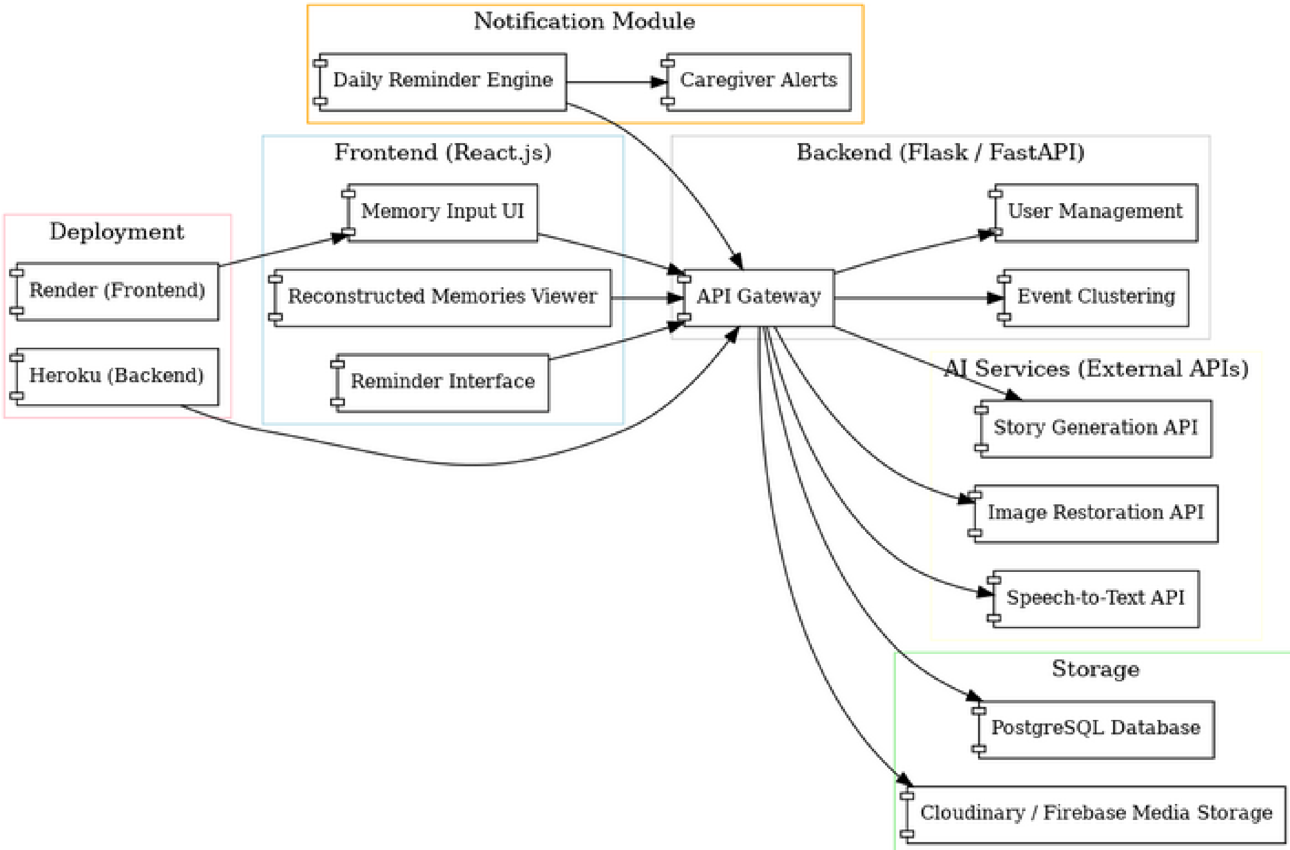


Fig 2.1.1: Component Diagram

ERD Diagram:

This Entity-Relationship Diagram (ERD) visualizes the structure of a database for ReMIND. It models key entities such as the adaptable User (specialized as Patient, Caregiver, and Admin) and the relationships that connect them. Patients, associated with Conditions, receive care and are managed by Caregivers. The operational core includes Chat Sessions for communication, composed of Messages and Media Files, with support for Reminders (which issue Notifications), and the capture of user Feedback. All critical activities are logged in an Audit Log, and the system includes functionality for processing media using AI Processing Tasks.

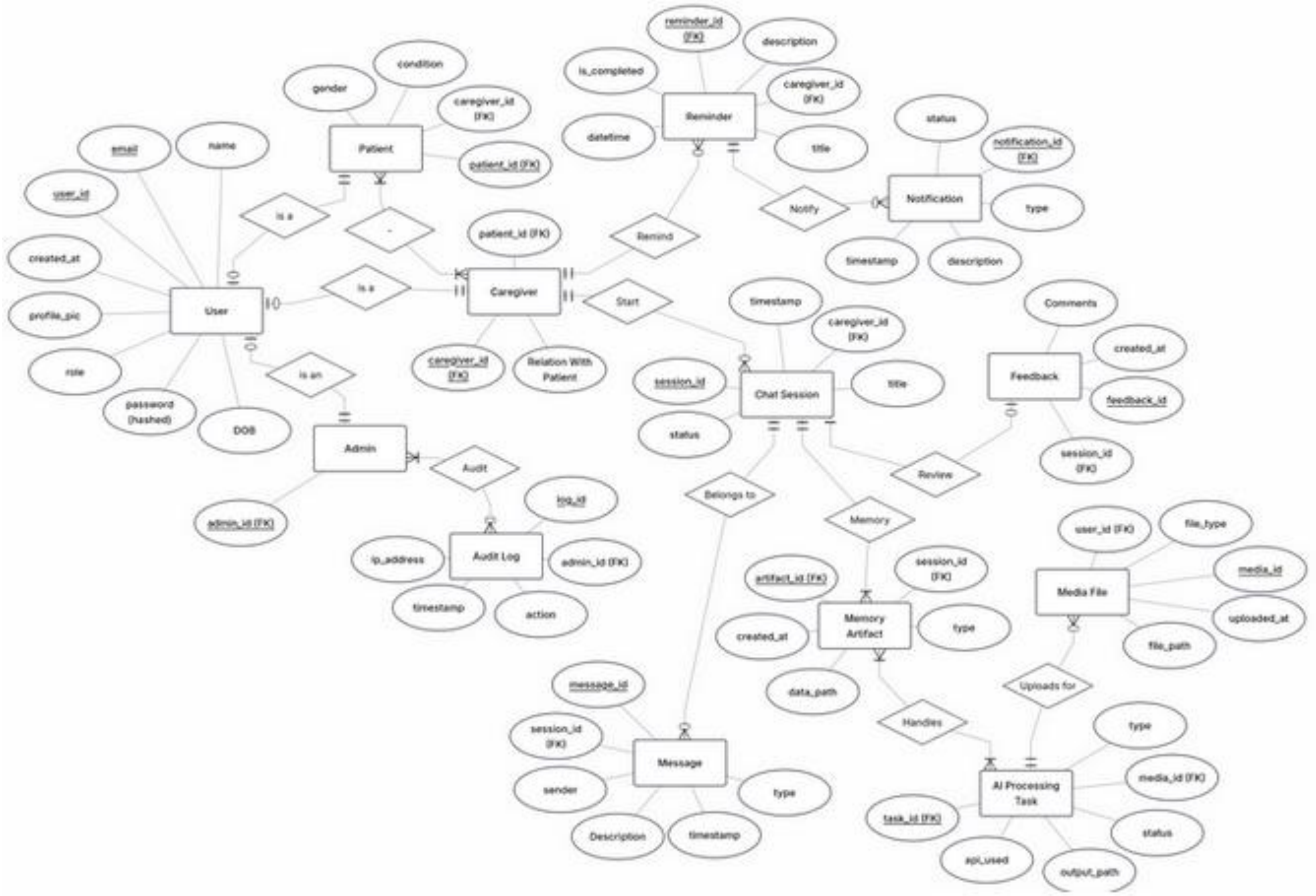


Fig 2.1.2: ERD Diagram

UML Class Diagram:
This UML Class Diagram models the structure of ReMIND. It organizes functionality into modules: Profiles, Audit Log, Caregiver & Patient, Chat, and AI & Media. The Profiles module defines User, specialized in Caregiver Profile, Patient Profile, and Admin Profile. The Caregiver & Patient Module manages Reminders and Notifications. The Chat Module handles Chat Sessions, Messages, and Feedback. Finally, the AI & Media Module defines Media, Memory Artifact, and AI Processing Task for handling media content.

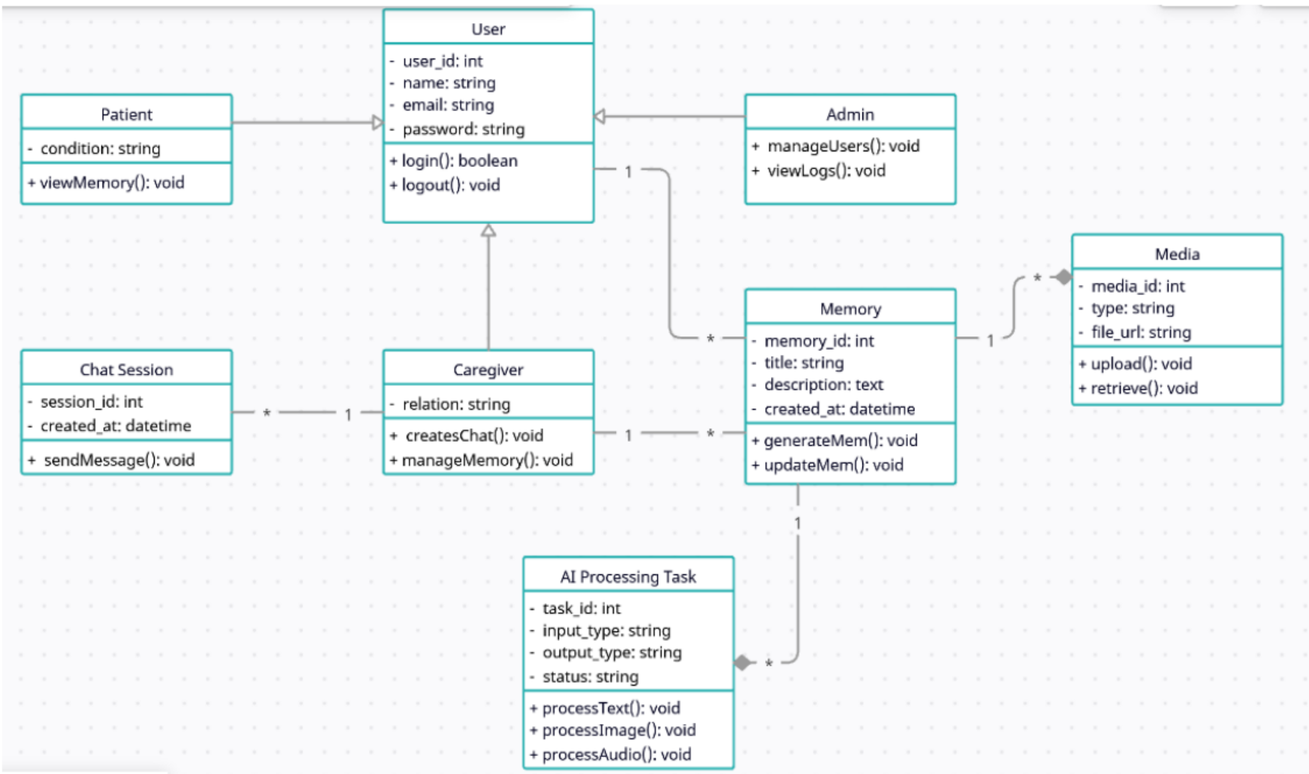


Fig 2.1.3: UML Class Diagram

Relational Schema:
This relational schema, based on the ReMIND-DATABASE ERD. It features a central User table, with specialized child tables: AdminProfile, CaregiverProfile, and PatientProfile, linked by foreign keys (FK) on the respective ID fields. The core interactions occur in the ChatSession table, which references CaregiverProfile and PatientProfile. Communication data is stored in Message and Media tables. Support functionality includes Reminder and Notification tables. All actions are tracked in the Auditlog table, while AIProcessingTask and MemoryArtifact handle media processing and retrieval.

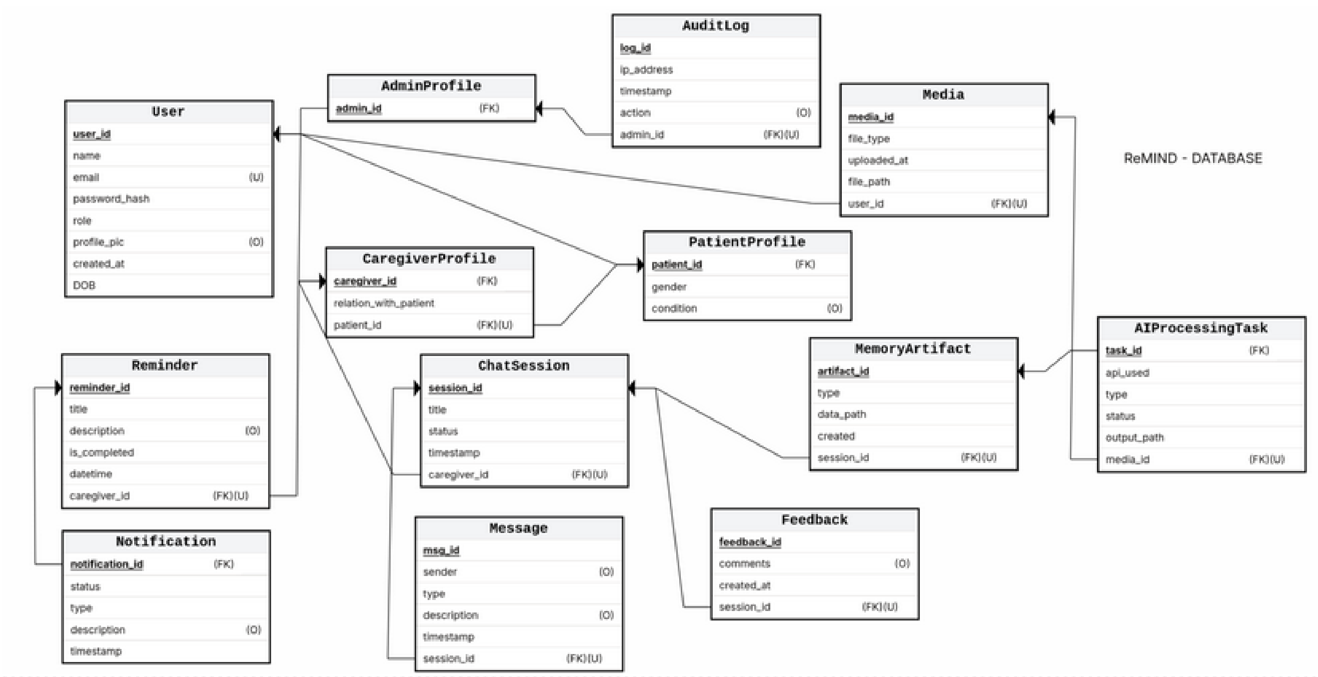


Fig 2.1.4: Relational Schema Diagram

Decision Table:

Rule	Input	Alternative Conditions	Action	Output
R1	User provides valid login credentials	Invalid credentials	Authenticate user, create session	Access granted / error message displayed
R2	Patient uploads multimedia input	Unsupported format or missing metadata	Accept input, store temporarily, trigger preprocessing	Acknowledgment / error message
R3	Uploaded file validated successfully	File corrupted or exceeds size limit	Proceed to AI preprocessing and enhancement	Process initiated / warning prompt
R4	Image input detected	No image detected	Trigger Real-ESRGAN enhancement and tagging	Enhanced image + metadata stored
R5	Audio input detected	No audio provided	Send to Whisper API for transcription	Transcript text returned
R6	Text or transcript available	Missing or empty text input	Send to GPT-4 for contextual understanding	Structured text/context response
R7	All media processed successfully	Any AI module fails	Aggregate data and reconstruct event	Reconstructed memory / retry message
R8	Narrative generation successful	AI fails to generate or timeout	Store reconstructed memory in repository	Text + voice response / system retry
R9	Reminder set by caregiver	Reminder missing or invalid	Save reminder in database	Reminder confirmation / validation error

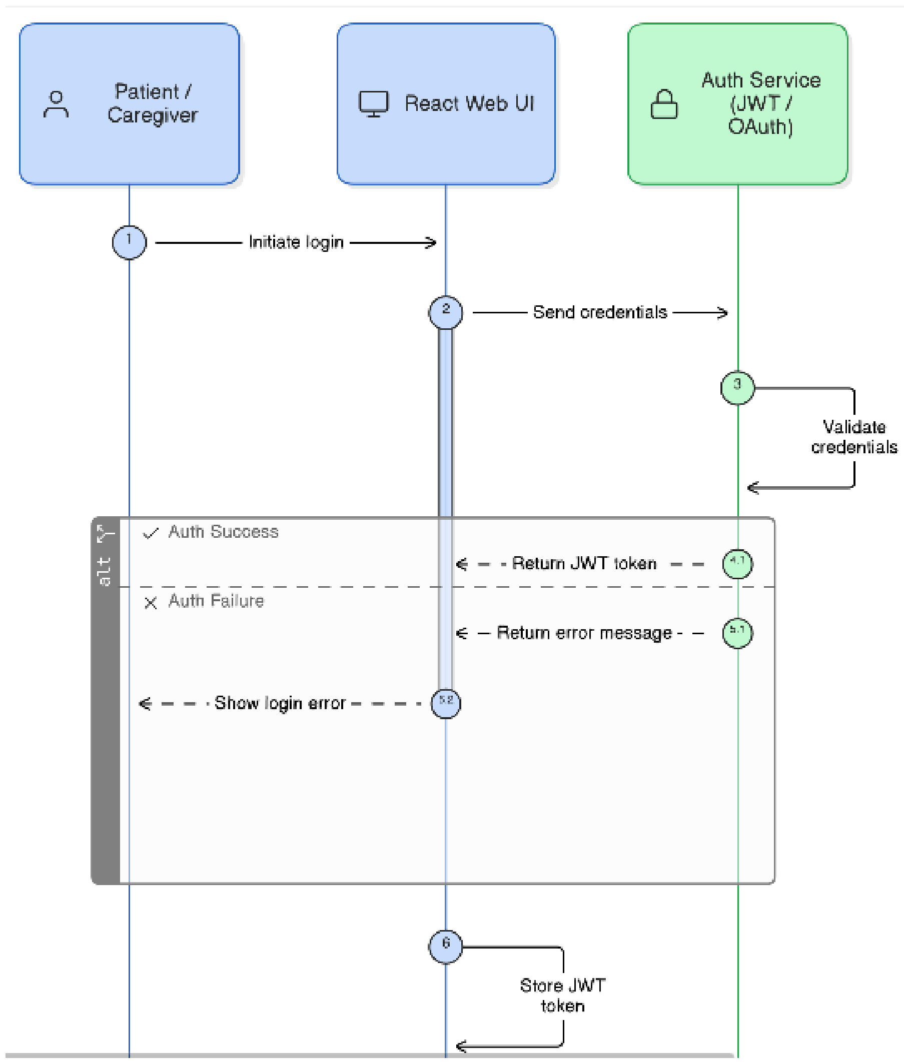
R10	Feedback submitted by patient/caregiver	Feedback skipped	Store feedback in log	Confirmation message / skip noted
R11	Admin monitors system logs	Logs not accessible	Display system status dashboard	Log summary / permission error

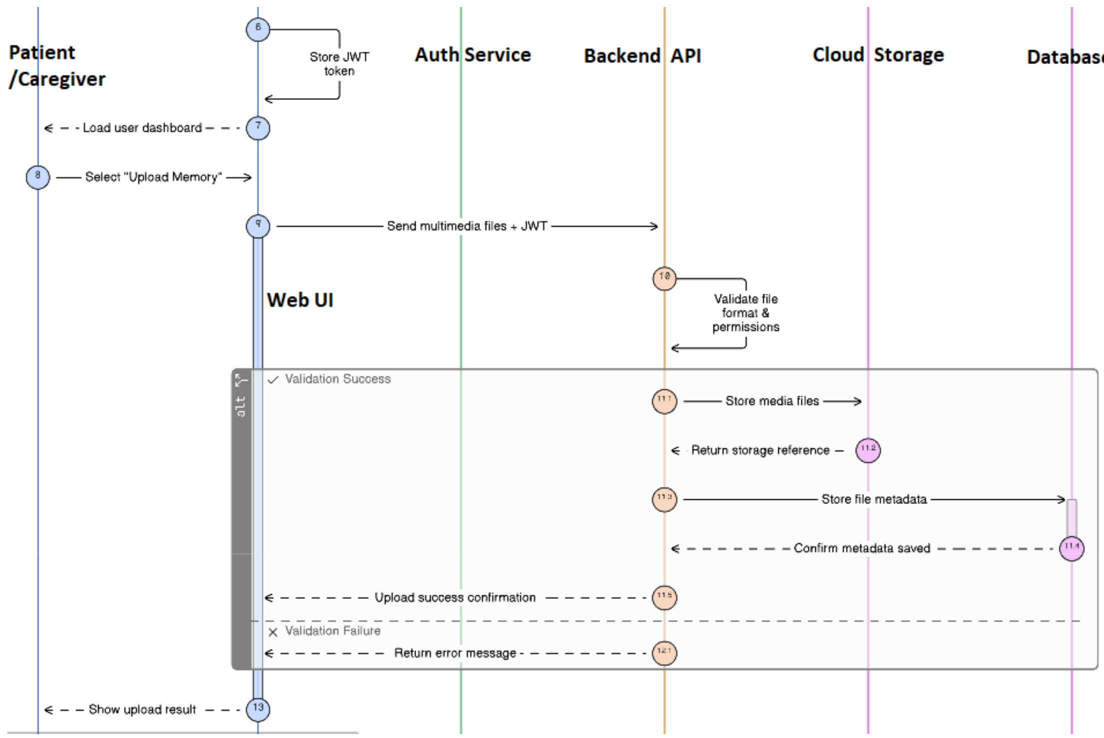
Table 2.1.5: Decision Table

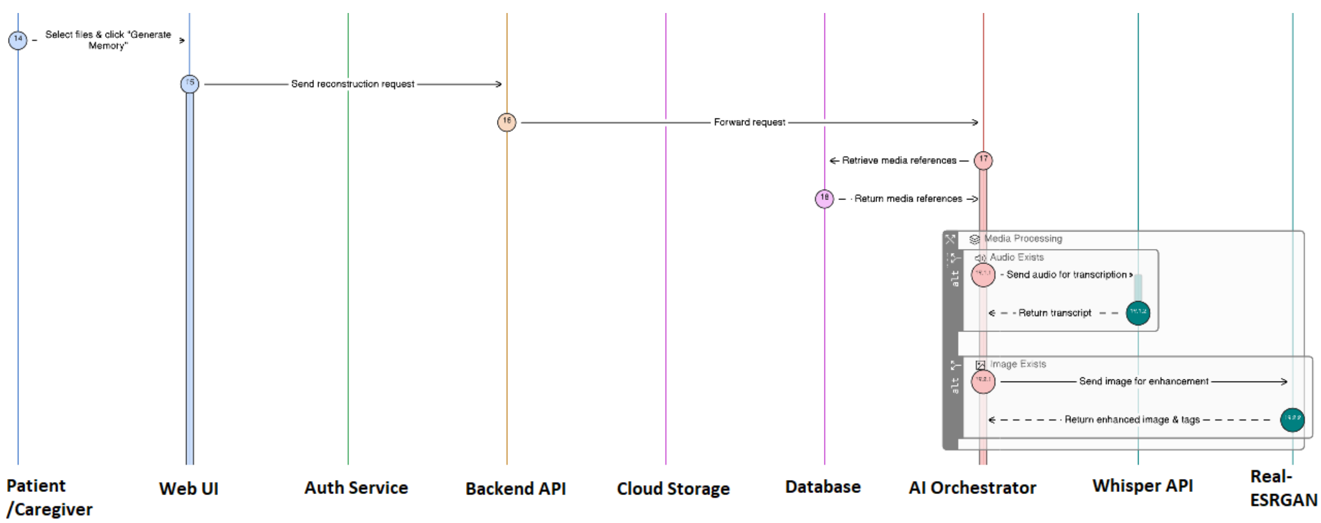
2.2 Component Interactions and Collaborations

Design Sequence Diagram:

The process flow begins with User Authentication, where the Frontend UI communicates with the Auth Service to validate credentials. Following login, the diagram maps the lifecycle of a memory: the Frontend sends data to the Backend (Flask/FastAPI), which coordinates with Cloud Storage (Cloudinary) for media and triggers AI Services (like Whisper API for speech-to-text or GPT-4 for analysis). These services process the data and return it to the Memory Reconstruction Engine. Finally, the rebuilt memory is saved to the Database and delivered back to the Frontend for the user to view.




Patient /Caregiver
Web UI
Auth Service
Backend API
Cloud Storage
Database
Store JWT token
Load user dashboard
Select "Upload Memory"
Send multimedia files + JWT
Validate file format & permissions
Validation Success
Store media files
Return storage reference
Store file metadata
Confirm metadata saved
Upload success confirmation
Validation Failure
Return error message
Show upload result


Select files & click "Generate Memory"
Send reconstruction request
Forward request
Retrieve media references
Return media references
Media Processing
Audio Exists
Send audio for transcription
Return transcript
Image Exists
Send image for enhancement
Return enhanced image & tags
Patient /Caregiver
Web UI
Auth Service
Backend API
Cloud Storage
Database
AI Orchestrator
Whisper API
Real-ESRGAN

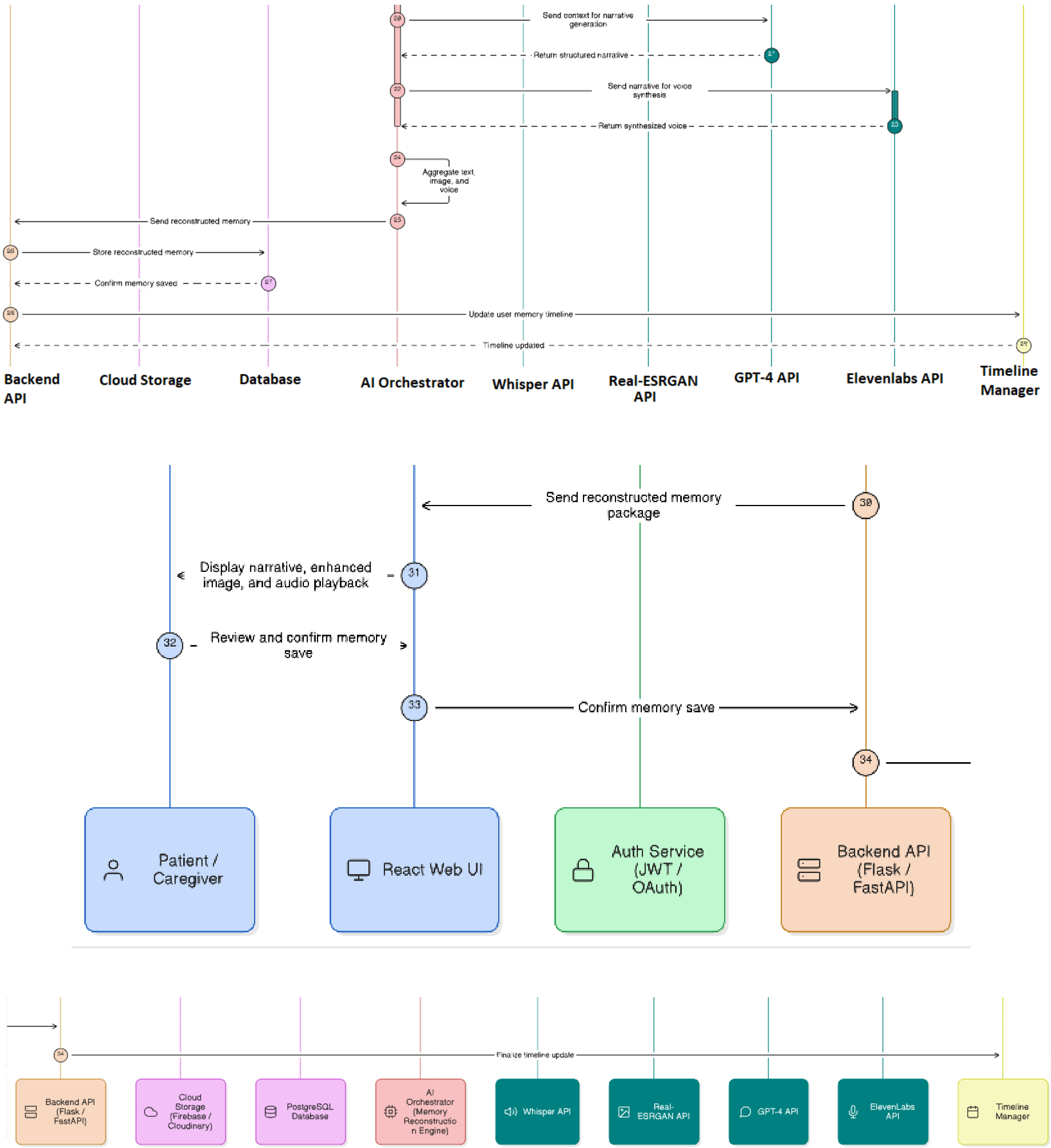


Fig 2.2.1: Sequence Diagram

Collaboration Diagram:

This Collaboration Diagram outlines the functional flow of the ReMIND system. It begins with the User initiating uploadMemory() (1) through the Frontend, which triggers sendJSON() (2) to the API Gateway. After the Auth Service performs authenticatesUser() (3), the MPC manages the Memory Reconstruction phase by identifying the input type (5) for the Text, Image, or Audio AI Engines.Once processed, the reconstructedOutput() (6) is sent to the Evaluation Engine to calculateAccuracy() (7) before being committed to the Database. The cycle completes as the system storeLogs() (8) and transmits a requestJSON() (9) back to the Frontend for final displayMemory() (10) via the Display Output module.

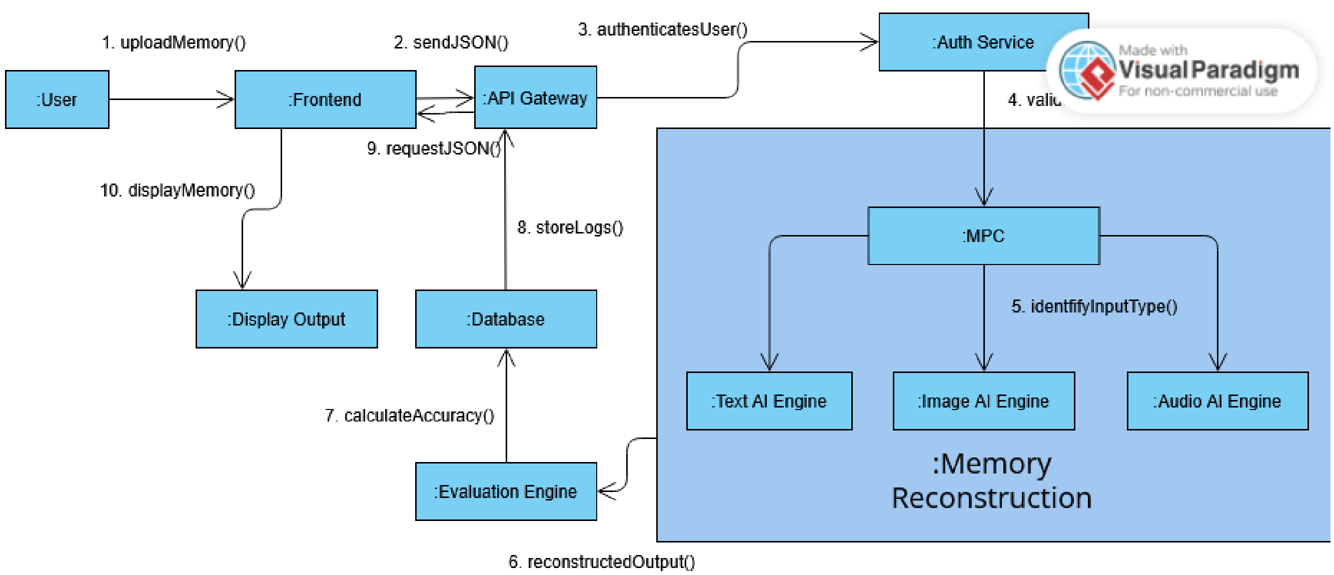


Fig 2.2.2: Collaboration Diagram

Activity Diagram:

This Activity Diagram models the sequential flow of actions a user takes within the ReMIND system, primarily focusing on memory handling. The process begins with Start and proceeds to Login. After successful login, the user must answer Security Questions (a combination of multiple-choice and input) for verification. Next, the user initiates Memory Input/Upload, likely submitting multimedia content or textual data. This input is then subjected to Memory Analysis/Processing, which transforms the raw data. The processed memory is then handled in Memory Storage & Tagging, where it is saved and categorized. Finally, the system executes Memory Retrieval/Display, presenting the reconstructed or stored memories to the user before the process reaches End.

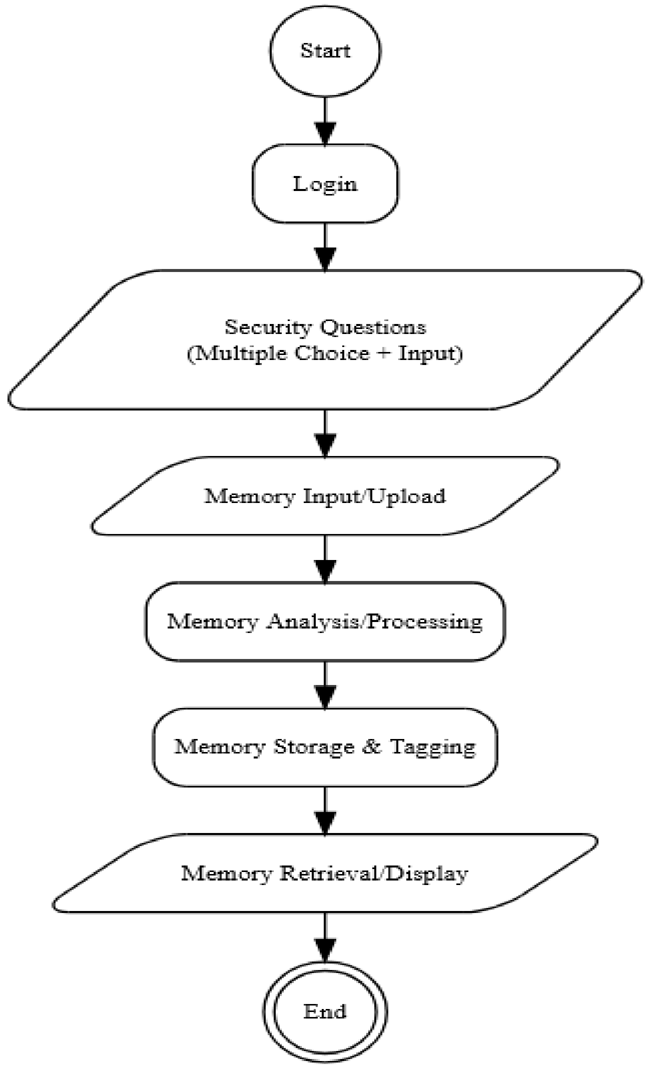


Fig 2.2.3: Activity Diagram

Event Table:

Event	Event Name	Trigger Type	Source	System Activity	Output
E1	User Login	External (User Action)	Patient / Caregiver / Admin	Authenticate credentials, create session	Dashboard or role-based access
E2	Media Upload	External (User Action)	Patient / Caregiver	Validate and store uploaded media	Temporary file storage + confirmation
E3	Data Preprocessing	Internal (System)	Backend	Cleans, formats, validates input data	Ready media for AI modules
E4	Image Enhancement	External (AI Service)	Real-ESRGAN	Enhances image and extracts context	Enhanced image file stored
E5	Audio Transcription	External (AI Service)	Whisper API	Converts audio to text, detects tone	Transcript returned
E6	Text Analysis	External (AI Service)	GPT-4	Interprets and structures text context	Semantic data created
E7	Event Reconstruction	Internal (AI Process)	AI Module	Combines multimodal data into cohesive memory	Structured narrative generated
E8	Voice Synthesis	External (AI Service)	ElevenLabs	Converts narrative text to natural audio	Voice file generated and stored
E9	Memory Storage	Internal (Database)	Backend → PostgreSQL	Saves reconstructed memory with metadata	Record created in memory repository
E10	Memory Retrieval	External (User Request)	Patient / Caregiver	Fetches requested memory or timeline	Displayed in frontend interface
E11	Reminder Creation	External (User Action)	Caregiver	Adds reminder entry in database	Confirmation sent to user
E12	Reminder Notification	Internal (Timed Event)	Notification Module	Sends scheduled alert to patient	Notification on UI
E13	Feedback Submission	External (User Action)	Patient / Caregiver	Logs ratings and suggestions	Stored in feedback log
E14	System Monitoring	Internal (Admin Action)	Administrator	Reviews logs, checks API integrations	Diagnostic report generated

Table 2.2.4: Event Table

Detailed DFD with Explanation:

DFD Level 1:

The DFD Level 1 diagram details the three main high-level processes that govern the system's operation: 3.0 User Authentication, 1.0 Data Processing, and 2.0 Memory Reconstruction. User Authentication (3.0) is the entry point, handling login/registration info from the Patient, Caregiver, and Admin entities and storing/retrieving user data from the Accounts DB. It also grants access permissions to Data Processing (1.0) and forwards session validation to Memory Reconstruction (2.0). Data Processing (1.0) receives multimedia uploads from the Patient and Caregiver; stores validated data in the Data Process DB and forwards the processed data to Memory Reconstruction (2.0). Finally, Memory Reconstruction (2.0) generates Memory Results/Feedback for the users, sends data analysis to AI Services, stores reconstructed memories in the Memory Artifacts DB, and provides reports/statistics to the Admin.

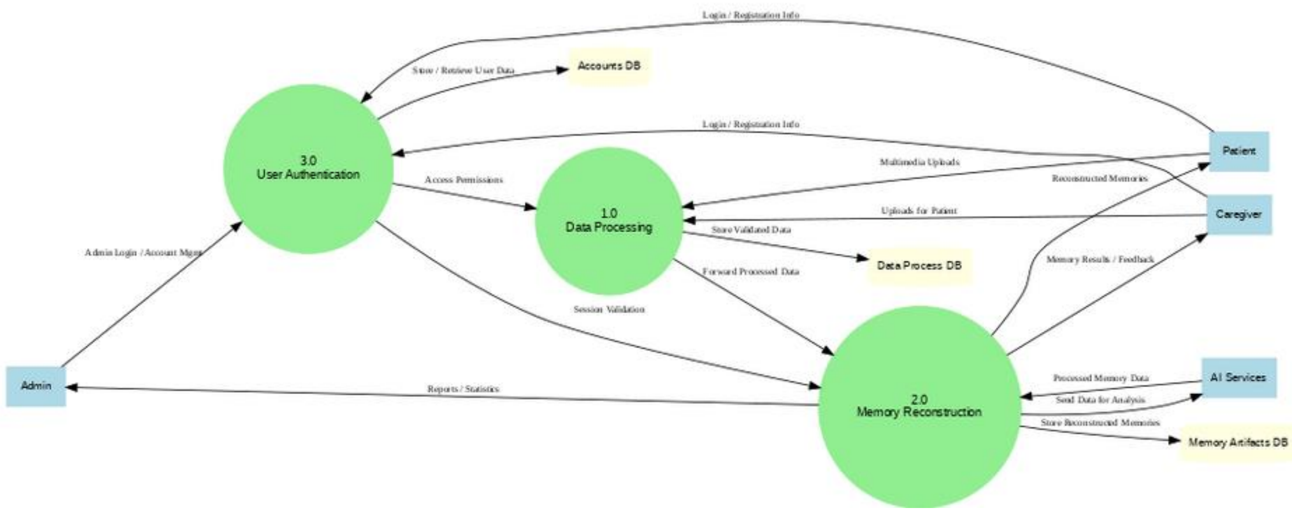


Fig 2.2.5: DFD Level 1

DFD Level 2:

The DFD Level 2 diagram elaborates on the three main processes of the system: 3.0 User Authentication, 1.0 Data Processing, and the now-decomposed 2.0 Memory Reconstruction. The Memory Reconstruction process (2.0) is broken down into two sub-processes: 2.0.1 API Calls & Integration and 2.0.2 Memory Verification. Data Processing (1.0) still receives uploads from the Caregiver and Patient and sends data to the Data Process DB, but it now forwards processed data to both the main Memory Reconstruction (2.0) process and 2.0.1 API Calls & Integration. API Calls & Integration (2.0.1) handles sending/receiving media from AI Services, logging API requests/responses in the APIs DB, and interacting with Data Processing (1.0). Finally, Memory Verification (2.0.2) receives data from the main Memory Reconstruction (2.0) process, verifies memories using multimedia from the Patient, stores the results in the Memory Artifacts DB, and sends verification results/feedback to the Admin.

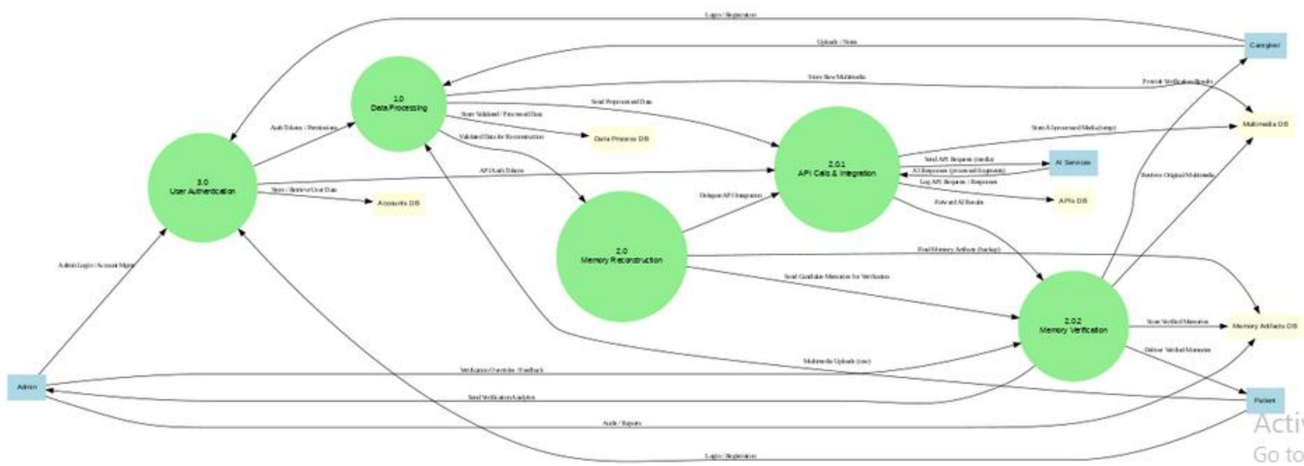


Fig 2.2.6: DFD Level 2

2.3 Design Reuse and Design Patterns

The development of ReMIND heavily leverages software reuse principles to ensure feasibility, reliability, and rapid prototyping within limited academic time and resources. Instead of developing low-level components from scratch, the system integrates externally available, well-tested frameworks, libraries, and commercial off-the-shelf (COTS) services, particularly in the domain of artificial intelligence.

At the application layer, widely adopted open-source frameworks such as React.js (frontend) and Flask/FastAPI (backend) are reused for interface development, API handling, and request orchestration. These frameworks provide standardized routing, state management, and security features, significantly reducing development complexity and risk.

At the intelligence layer, ReMIND reuses pre-trained generative and perceptual AI models offered through commercial APIs, including GPT-4 for narrative generation, Whisper for speech transcription, Real-ESRGAN for image enhancement, and ElevenLabs for voice synthesis. This reuse eliminates the need for custom model design, training, and optimization, making the project technically feasible while ensuring high-quality outputs.

In the data layer, existing database management systems (PostgreSQL) and cloud-based media storage services are reused for persistent storage, backup, and access control.

Additionally, development tools such as GitHub for version control, Postman for API testing, and Figma for UI prototyping are reused as part of standard software engineering practice.

Overall, approximately 30–40% of the system relies on reused components, while 60–70% represents original work focused on system integration, workflow design, data modeling, and user-centric application logic.

2.4 Technology Architecture

Data and Processing Characteristics

ReMIND collects and manages:

- User credentials and role information
- Multimedia inputs (text, images, audio)
- AI-generated narratives and voice outputs
- Memory timelines and reminder schedules
- User feedback and system logs

Processing is online and event-driven, with asynchronous AI calls to external services. Each user action triggers a transaction that results in AI processing, data persistence, and user-facing output.

Technology Stack and Platforms

- Python, JavaScript are Programming languages used.
- React.js and CSS3 are used to create Front-end.
- Flask/FastAPI is used to create Back-end.
- OpenAI GPT-4, Whisper, ElevenLabs, Real-ESRGAN are our COTS AI Services.
- PostgreSQL is used to manage databases.
- Cloud-based media storage (Firebase/Cloudinary) is used for Media storage and management.
- External cloud infrastructure (e.g., Render / cloud servers) for hosting.
- Personal laptops for development, university GPU systems for AI integrated workloads as hardware.
- Our system is supported on all browsers on any desktop or mobile device for an end-user interface.

Interface and Network Architecture

The system provides a **browser-based user interface** accessible over the **Internet (WAN)**. Communication between frontend, backend, and AI services occurs via secure RESTful APIs. The architecture supports remote access for caregivers and administrators without requiring specialized client software.

2.5 Architecture Evaluation

a. Frontend Technology Selection

Selected Technology: React.js with CSS3

Reason for Selection:

React.js was selected because it provides a component-based architecture that simplifies the development of dynamic and interactive user interfaces. Since ReMIND requires an elderly-friendly UI with real-time updates and multimedia interaction, React enables efficient state management and responsive rendering. It also integrates easily with REST APIs and modern JavaScript tools.

Advantages:

- Component-based architecture enables modular UI development
- Strong ecosystem and large developer community
- Efficient virtual DOM rendering improves performance
- Easy integration with backend APIs and cloud services
- Supports responsive and accessible UI design

Disadvantages:

- Requires familiarity with modern JavaScript concepts
- Slightly steeper learning curve compared to basic HTML frameworks

Alternative Considered: Angular

Angular provides a full MVC framework but introduces greater complexity and heavier structure. React was preferred because it is lighter, easier to integrate with APIs, and faster to prototype for academic projects.

b. Backend Framework Selection

Selected Technology: Flask / FastAPI (Python)

Reason for Selection:

Flask and FastAPI were chosen due to their lightweight architecture and strong compatibility with AI and machine learning libraries written in Python. Since ReMIND integrates multiple AI APIs such as

GPT-4, Whisper, and Real-ESRGAN, Python-based frameworks allow seamless integration and rapid API development.

Advantages:

- Lightweight and flexible architecture
- Easy integration with AI and data processing libraries
- REST API development is simple and efficient
- FastAPI provides high performance and automatic documentation

Disadvantages:

- Less built-in functionality compared to larger frameworks
- Requires manual setup for some components

Alternative Considered: Django

Django is a full-stack framework with built-in modules, but it introduces unnecessary overhead for a prototype system. Flask/FastAPI were preferred because they allow faster development and simpler microservice architecture.

c. AI Model and API Selection

The ReMIND system relies on commercial off-the-shelf AI services instead of training custom models, enabling faster development and higher-quality outputs.

GPT-4 (Text Processing and Narrative Generation)

Reason for Selection:

GPT-4 was selected because it provides advanced natural language understanding and contextual text generation, which is essential for reconstructing meaningful narratives from fragmented memory inputs.

Advantages:

- Highly accurate contextual language generation
- Strong semantic reasoning capabilities
- Reliable API with extensive documentation

Disadvantages:

- API usage costs
- Dependence on external service availability

Alternative: Open-source models (e.g., LLaMA)

Open-source models require high GPU resources and extensive training pipelines, which are not feasible under the project's resource constraints.

Whisper (Speech Recognition)

Reason for Selection:

Whisper provides robust speech-to-text transcription across multiple accents and noisy audio conditions, making it suitable for elderly users who may have varied speaking patterns.

Advantages:

- High transcription accuracy
- Supports multiple languages and accents
- Works well with low-quality recordings

Disadvantages:

- Requires API calls or computational resources

Alternative: Google Speech-to-Text

Although accurate, Whisper provides better open integration and research reliability for AI experiments.

Real-ESRGAN (Image Enhancement)

Reason for Selection:

Real-ESRGAN improves the quality of old or low-resolution photographs, which is essential for reconstructing memories from historical images.

Advantages:

- High-quality image restoration
- Effective for noisy or degraded photos
- Open-source and research-supported

Disadvantages:

- Computationally expensive

Alternative: OpenCV super-resolution methods

Traditional methods offer lower quality restoration compared to modern GAN-based models.

ElevenLabs (Voice Synthesis):

Reason for Selection:

ElevenLabs was chosen because it produces highly realistic speech synthesis, enabling reconstructed memories to be narrated in a natural voice.

Advantages:

- Natural-sounding voice generation
- Multiple voice styles
- Easy API integration

Disadvantages:

- Usage cost limits on free plans

Alternative: Google Text-to-Speech

Although widely used, ElevenLabs provides more realistic emotional voice synthesis, which aligns better with the emotional goals of ReMIND.

d. Database Selection

Selected Technology: PostgreSQL

Reason for Selection:

PostgreSQL was selected due to its reliability, scalability, and strong support for relational data modeling. The system requires structured storage for users, memories, timelines, reminders, and metadata.

Advantages:

- Open-source and reliable
- Strong relational data integrity
- Advanced querying capabilities

Disadvantages:

- Requires proper schema design

Alternative: MongoDB

MongoDB offers flexible schema design but is less suitable for structured relational data such as user relationships and memory timelines.

e. Media Storage Selection

Selected Technology: Firebase / Cloudinary

Reason for Selection:

These services provide cloud-based storage optimized for multimedia files, enabling efficient storage and retrieval of images and audio files used by the AI processing modules.

Advantages:

- Optimized media delivery
- CDN support for fast access
- Easy API integration

Disadvantages:

- Free tier storage limits

Alternative: AWS S3

AWS provides powerful storage but requires more complex configuration and management.

f. Deployment Infrastructure

Selected Platforms: Render (Frontend) and Heroku (Backend)

Reason for Selection:

These platforms were selected because they provide simple deployment pipelines, free-tier hosting options, and seamless GitHub integration, which are suitable for academic prototype systems.

Advantages:

- Easy deployment and CI/CD integration
- Free-tier hosting options
- Supports modern web stacks

Disadvantages:

- Cold start delays in free tiers
- Limited computing resources

Alternative: AWS / Google Cloud

Although more powerful, they require advanced configuration and may exceed the project's limited budget.

3. Detailed/Component Design

Deployment Diagram:

The ReMIND system is accessed through a user device (browser) where the React.js frontend is deployed, providing interfaces for memory input, viewing, and interaction. User requests are sent via HTTPS to the backend server (FastAPI/Flask on Heroku), which handles authentication, processing, and system logic. The backend communicates with external AI services (GPT-4, Whisper, ElevenLabs, Real-ESRGAN) for memory reconstruction. Structured data is stored in a PostgreSQL database, while multimedia files are managed through cloud storage (Firebase/Cloudinary). After processing, the backend returns reconstructed memories and notifications to the frontend. This architecture ensures a scalable, cloud-based, and AI-driven memory reconstruction system.

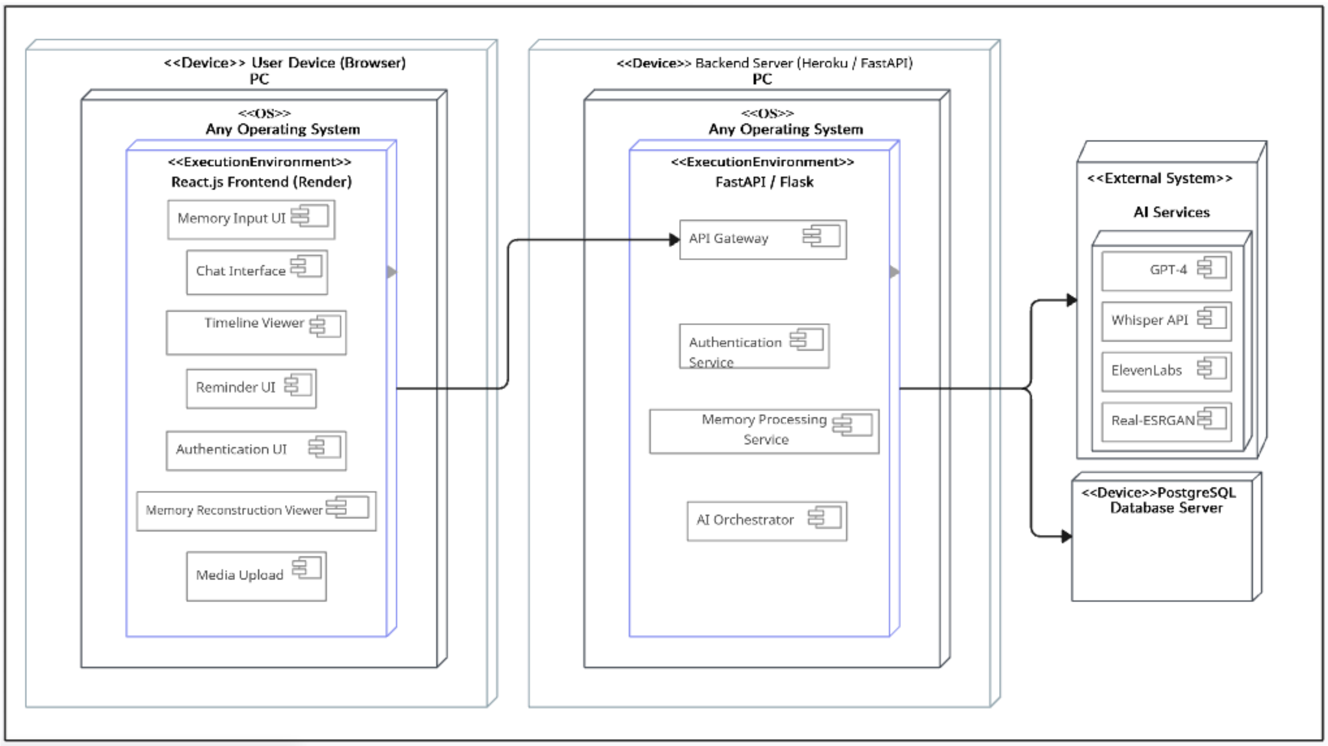


Fig 3.1: Deployment Diagram

Petri Net:

The Petri Net diagram models the memory reconstruction workflow of ReMIND. It represents the flow of multimedia input through validation, storage, and parallel AI processing stages. Separate transitions handle text extraction, image enhancement, and audio transcription simultaneously. These outputs are then aggregated to generate a reconstructed memory, which is stored and displayed to the user. The Petri Net effectively captures concurrency and synchronization within the system.

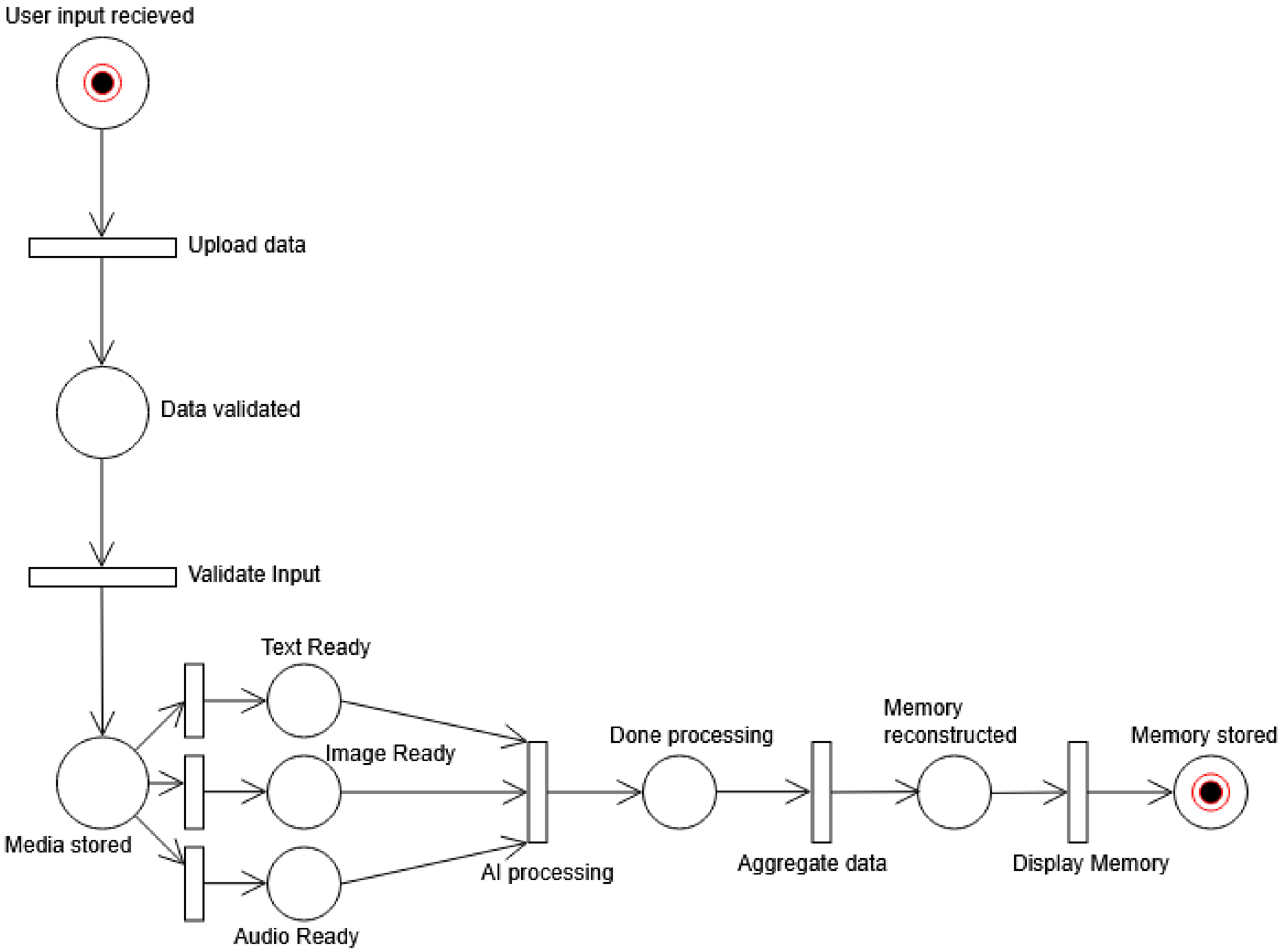


Fig 3.2: Petri Net Diagram

3.1 Component-External Entities Interface

The ReMIND system communicates with several external entities to provide AI-assisted memory reconstruction services. The Backend API layer acts as the central controller and exchanges data with external AI services such as GPT-4 for narrative generation, Whisper for speech-to-text transcription, ElevenLabs for voice synthesis, and Real-ESRGAN for image enhancement. Structured data such as user profiles, memories, and metadata are stored in the PostgreSQL database, while multimedia files are managed through Firebase or Cloudinary cloud storage. Notification or email services may also be connected to reminders and alerts. These interfaces use secure REST APIs and HTTPS protocols to ensure reliable and protected communication between ReMIND and third-party systems.

For Diagram, Check Figure 2.1.1 in Section 2.1 which is Component Diagram.

3.2 Component-Human Interface

The human interface of ReMIND is designed to be simple, accessible, and elderly-friendly for patients and caregivers. Major screens include the Landing Page, Login/Register Screen, Security Questions Screen, Dashboard, Memory Input Screen, Chat Interface, Timeline Viewer, Reconstructed Memory Viewer, Admin Dashboard, and Notification/Reminder Screen. Users can upload text, audio, or images, view generated memories, and interact through a chat-based interface. The interface follows HCI principles such as large readable fonts, high contrast colors, minimal navigation complexity, clear buttons, responsive layout, and consistent feedback messages. Accessibility and ease of use were prioritized to support elderly users and non-technical caregivers.

For Details of Design check further sections.

4. Screenshots/Prototype

4.1 Workflow

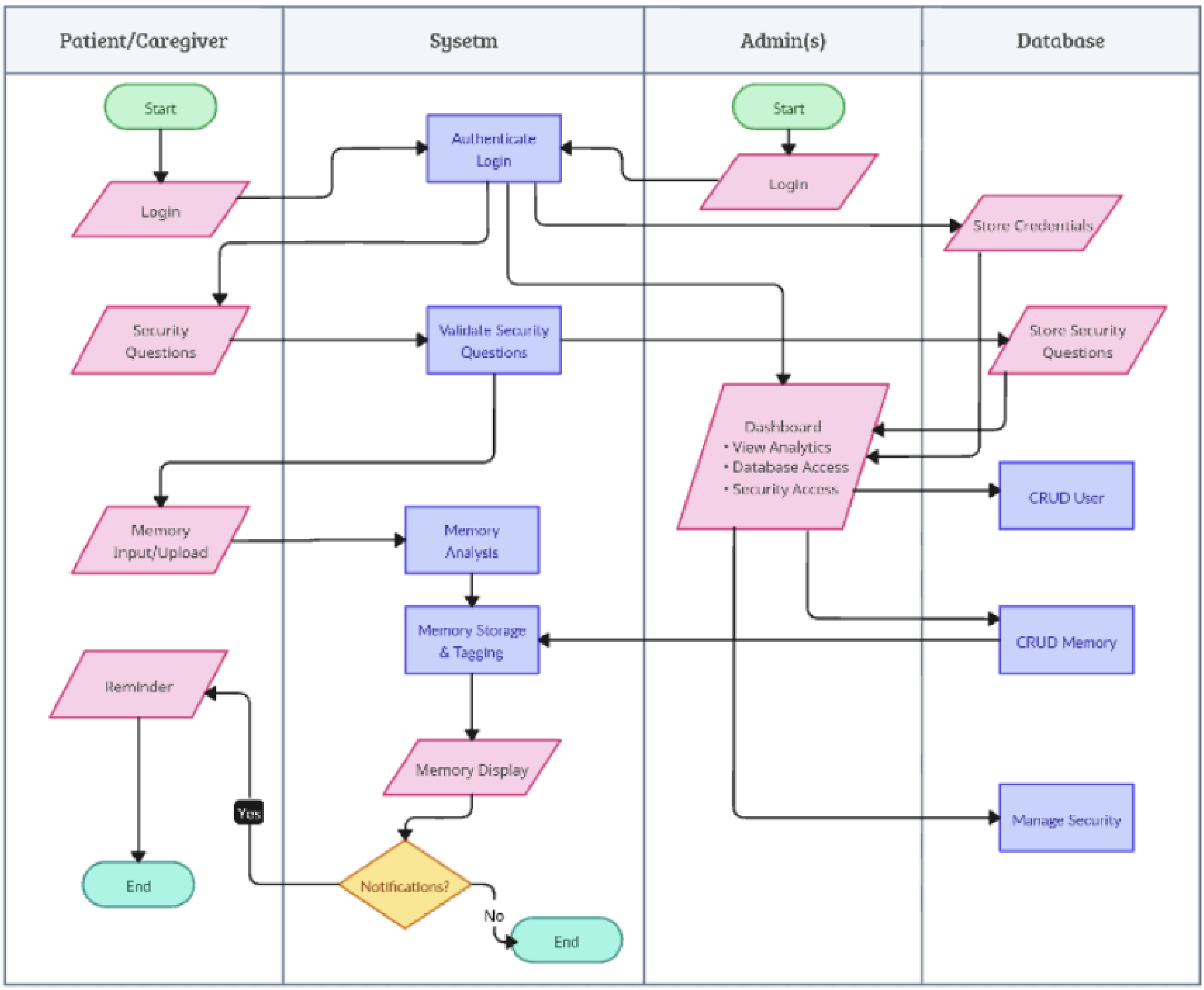


Fig 4.1: Swim-Lane Workflow Diagram

The workflow of ReMIND begins when a Patient/Caregiver or an Admin initiates the process through a web-based interface. This start phase requires a secure Login, which triggers the System to perform an authentication check. To ensure robust security, the user must then answer Security Questions, which the System validates against records in the Database. This Database layer is central to the operation, securely storing credentials and security question data to facilitate these initial access steps.

Once the user is successfully authenticated, the Patient/Caregiver can provide a Memory Input/Upload consisting of text, images, or audio. The System immediately moves this data into a Memory Analysis phase, where fragmented inputs are processed for contextual meaning. Following analysis, the System performs Memory Storage and Tagging, interacting with the Database to ensure the data is categorized correctly. For the Administrative side, the admin manages the platform via a Dashboard that allows for viewing analytics and performing CRUD operations on both Users and Memories within the Database. The admin also handles the Manage Security function to maintain system integrity.

The final stage of the workflow involves the System generating a Memory Display for the user to review. After the memory is presented, the System checks the status of Notifications. If the user has notifications enabled, the process triggers a Reminder for the Patient/Caregiver to support daily memory reinforcement before reaching the end of the session. If notifications are not enabled, the workflow proceeds directly to the end, concluding the cycle of memory reconstruction and administrative oversight.

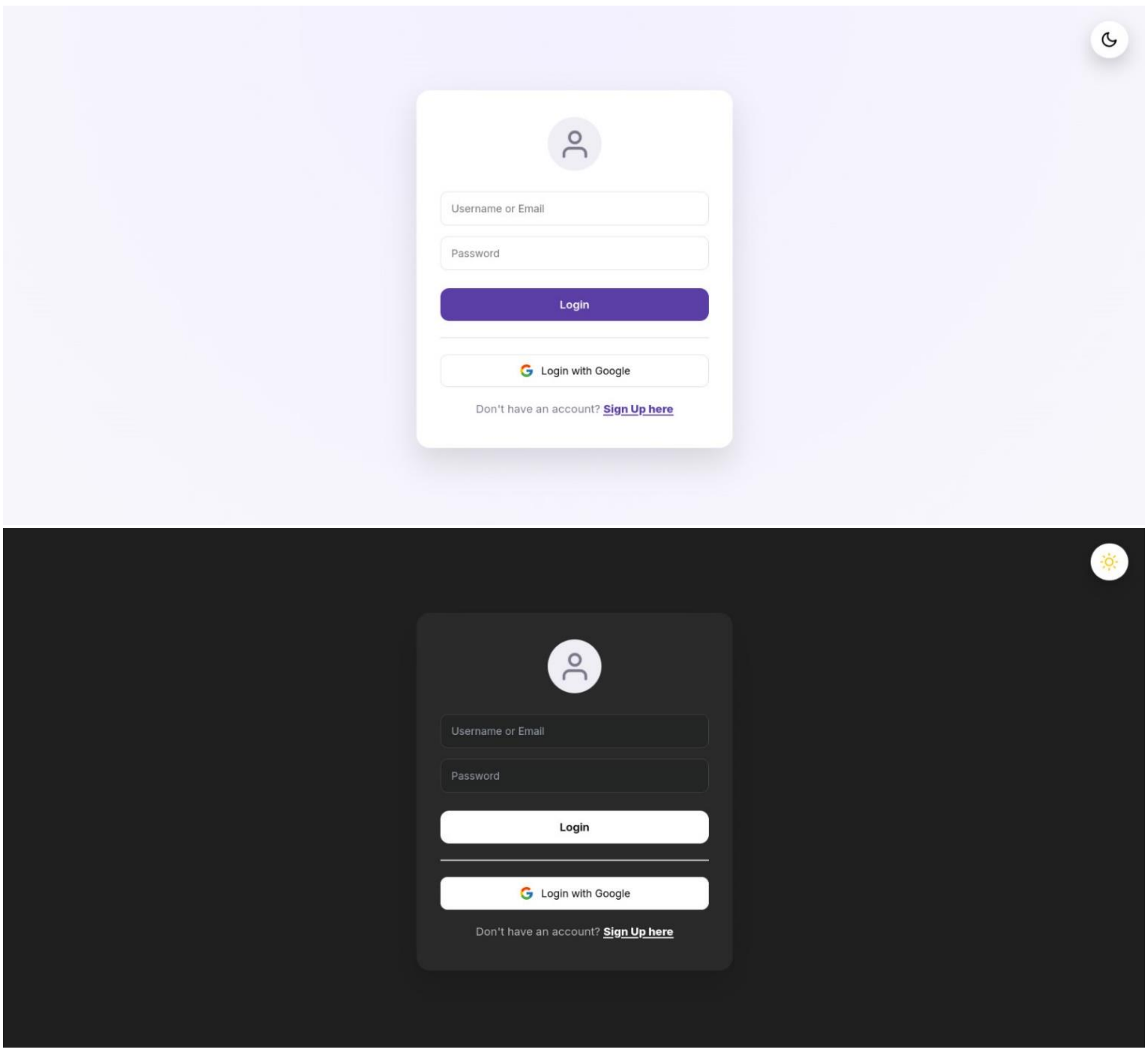


Fig 4.2.1: Login System is Light and Dark Mode

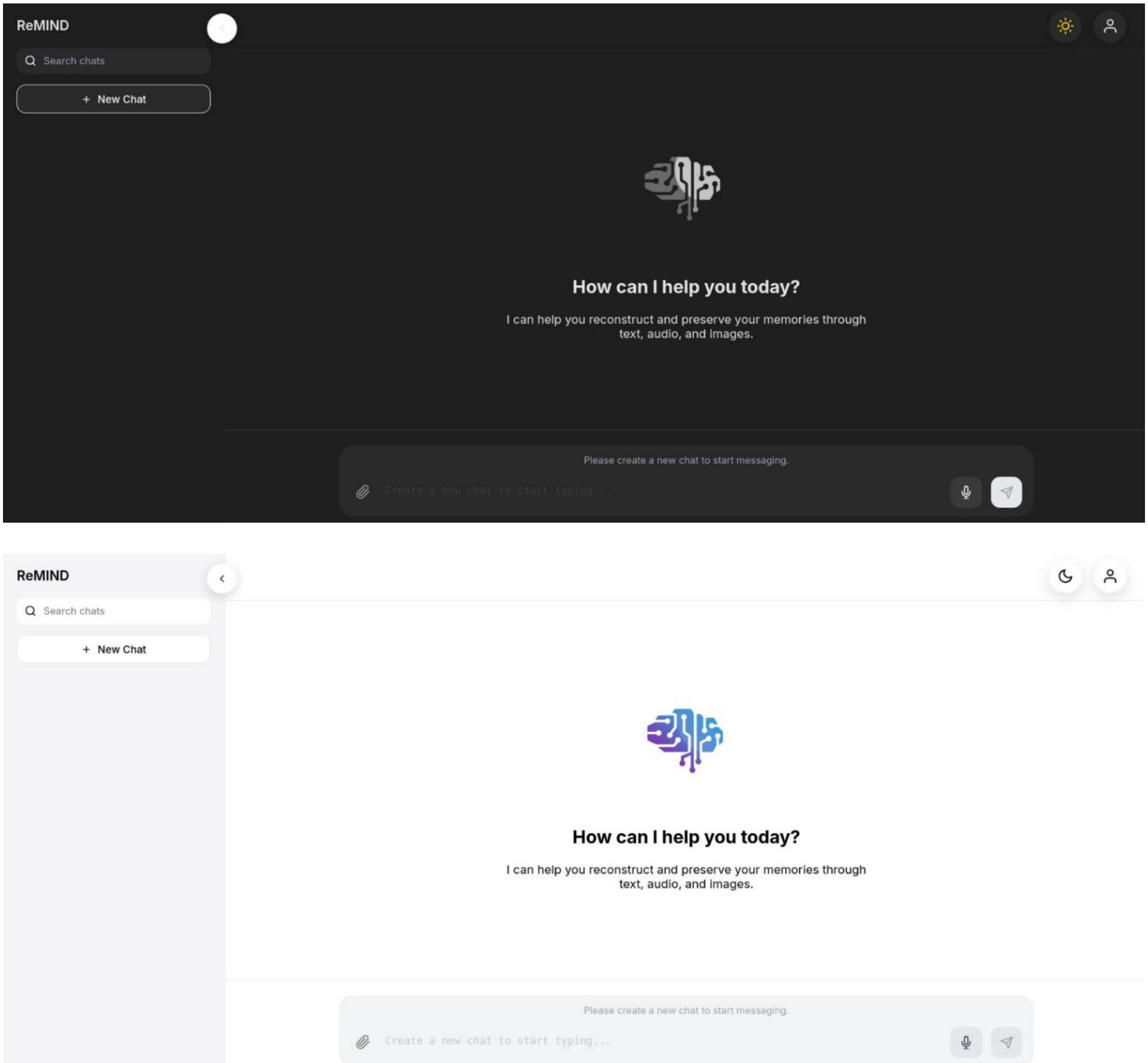


Fig 4.2.2: Main Chat UI in Dark and Light Mode

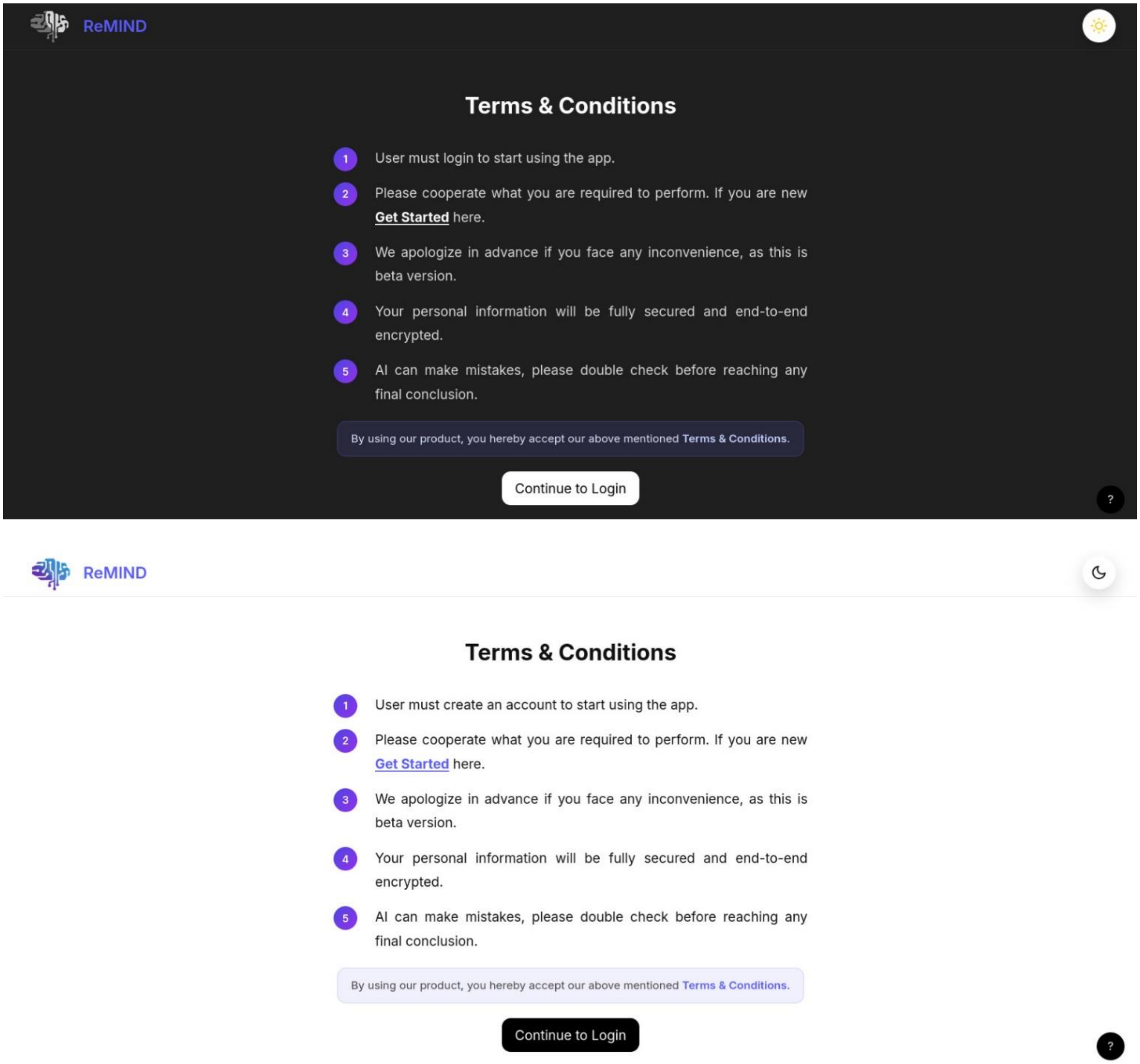


Fig 4.2.3: Terms & Conditions page (Landing) in Dark and Light mode.

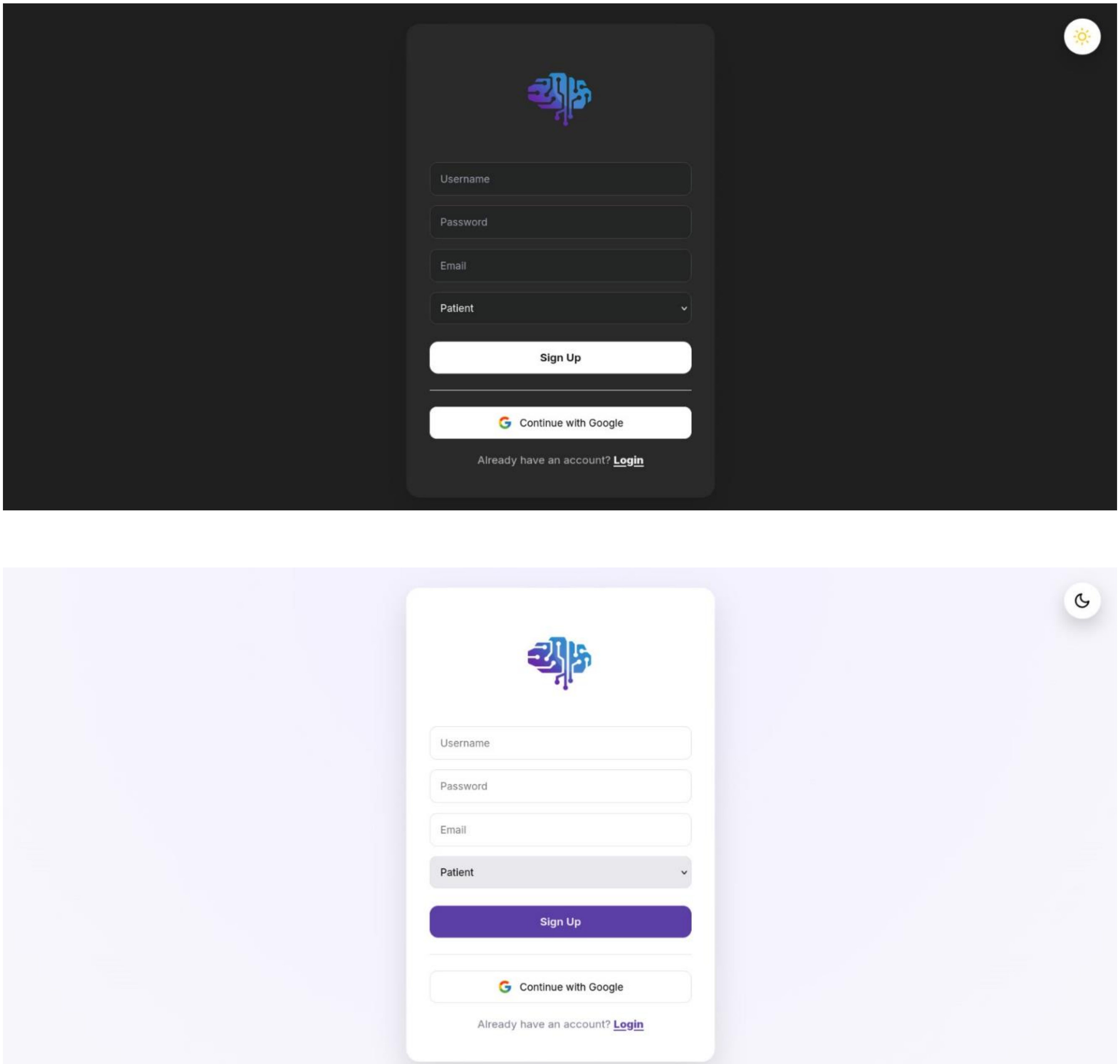


Fig 4.2.4: Signup System in Dark and Light Mode.

5. Test Specification and Results

5.1 Test Case Specification

Table 5.1.1: TC-1 (User Login Authentication)

Identifier	TC-1
Related requirements(s)	SRS Section 2.2 Functional Description – User Authentication
Short description	Verify that a registered user can successfully log into the system.
Pre-condition(s)	User account must already exist in the PostgreSQL database.
Input data	Username: any_username Password: any_password
Detailed steps	1. Open ReMIND web application. 2. Navigate to Login page. 3. Enter valid username and password. 4. Click the Login button.
Expected result(s)	User is authenticated and redirected to the main dashboard.
Post-condition(s)	User sessions are created and stored using JWT authentication.
Actual result(s)	User successfully logged in and loaded the dashboard.
Test Case Result	PASS

Table 5.1.2: TC-2 (Text Memory Input)

Identifier	TC-2
Related requirements(s)	SRS Section 2.1 – Memory Input Module
Short description	Verify that users can submit textual memory data successfully.
Pre-condition(s)	Users must be logged into the system.
Input data	Text memory typed input: “This is sample text memory input.”
Detailed steps	1. Login to the system. 2. Navigate to the Chat Box. 3. Type in the Memory Text. 4. Click Send.
Expected result(s)	Memory text is stored in the PostgreSQL database with metadata.
Post-condition(s)	

	Memory artifact entry created in database.
Actual result(s)	Memory text was stored successfully in the database.
Test Case Result	PASS

Table 5.1.3: TC-3 (Image Upload)

Identifier	TC-3
Related requirements(s)	SRS Section 2.1 – Multimedia Input
Short description	Verify that users can upload images for memory reconstruction.
Pre-condition(s)	Users must be logged into the system.
Input data	Any Image File is Supported Format: Image.png
Detailed steps	1. Login to System. 2. Navigate to Upload Button, Select Image. 3. Upload Image File, Type-in Optional Text. 4. Click the Send Button.
Expected result(s)	Image stored in Cloudinary, Metadata stored in PostgreSQL.
Post-condition(s)	Image metadata stored in database.
Actual result(s)	The image was uploaded successfully and metadata generated.
Test Case Result	PASS

Table 5.1.4: TC-4 (Audio Upload)

Identifier	TC-4
Related requirements(s)	SRS Section 2.1 – Audio Input
Short description	Verify that users can upload Audio Files for memory reconstruction.
Pre-condition(s)	Users must be logged into the system.
Input data	Any Audio File is Supported Format: Audio.wav
Detailed steps	1. Login to System. 2. Navigate to Upload Button, Select Image. 3. Upload Audio File, Type-in Optional Text. 4. Click the Send Button.
Expected result(s)	Audio File stored in Cloudinary, Metadata stored in PostgreSQL.

Post-condition(s)	Audio File metadata stored in database.
Actual result(s)	Audio File was uploaded successfully and metadata generated.
Test Case Result	PASS

Table 5.1.5: TC-5 (Frontend → Backend API Communication)

Identifier	TC-5
Related requirements(s)	SRS Section 3.1 – Application Architecture
Short description	Verify successful communication between frontend and backend API endpoints.
Pre-condition(s)	Backend server running and API endpoints configured.
Input data	Any API request: POST /api/memory
Detailed steps	1. Send an API request from the frontend. 2. Backend receives requests. 3. Backend validates requests. 4. The backend sends a response.
Expected result(s)	Backend returns status ‘200 OK’ with response data.
Post-condition(s)	Data stored and response returned to frontend.
Actual result(s)	API returned the correct response.
Test Case Result	PASS

Summary of Test Results

Table 5.2: Summary of Test Results

Module Name	Test cases run	Number of defects found	Number of defects corrected so far	Number of defects still need to be corrected
Authentication	TC1	3	3	0
Memory Input	TC2	0	0	0
Image Upload	TC3	1	1	0
Audio Upload	TC4	1	1	0
API Communication	TC5	2	2	0
Complete System	5	7	7	0

6. Revised Project Plan

The ReMIND project is currently in the development phase following the completion of the design and architecture stages. Initial progress has focused on implementing the frontend interface, backend API structure, and database schema, which form the foundation for integrating AI services in later phases.

At the current stage, the development team has begun implementing the React-based user interface and Flask/FastAPI backend architecture, along with the PostgreSQL database structure for storing user information, memory artifacts, and metadata. Basic multimedia input functionality such as text entry, image uploads, and audio uploads has also been initiated.

However, several advanced modules including AI-based speech transcription, image enhancement, memory reconstruction, and voice synthesis are still pending implementation. These components depend on the integration of external AI APIs such as Whisper, GPT-4, Real-ESRGAN, and ElevenLabs.

Future development phases will focus on completing these AI integrations, implementing the reminder system, enabling caregiver collaboration features, and deploying the full system on cloud infrastructure using Render and Heroku.

Table 6.1: Project Completion Status

Module Name	Status
User Authentication Module (Login / Signup / Session Management)	Fully Implemented
User Interface Module (React Frontend UI, Navigation, Dashboard)	Fully Implemented
Memory Input Module (Text Memory Entry)	Fully Implemented
Multimedia Upload Module (Image Upload)	Fully Implemented
Audio Upload Module (Voice Recording / Upload)	Fully Implemented
Backend API Module (Flask/FastAPI Endpoints)	Partially Implemented
Database Management Module (PostgreSQL Data Structuring)	Partially Implemented
Metadata Management Module (JSON Metadata for Text/Image/Audio)	Partially Implemented
Speech Processing Module (Whisper Transcription)	Not Implemented Yet
Image Enhancement Module (Real-ESRGAN Processing)	Partially Implemented
Memory Reconstruction Module (GPT-4 Narrative Generation)	Not Implemented Yet
Voice Synthesis Module (ElevenLabs TTS)	Partially Implemented
Event Clustering and Timeline Generation Module	Not Implemented Yet
Reminder and Notification Module	Not Implemented Yet
Caregiver Collaboration Module	Not Implemented Yet
Cloud Media Storage Module (Firebase / Cloudinary)	Not Implemented Yet
System Deployment Module (Render / Heroku)	Not Implemented Yet
System Security Module (JWT / Encryption / Access Control)	Partially Implemented
Testing and Validation Module	Partially Implemented

Complete System	**In Development**

Task / Module	Start Date	End Date	Owner
Project Planning & Requirement Review	2026-01-01	2026-01-10	Team
System Architecture & Design Finalization	2026-01-05	2026-01-15	Team
Frontend Development (React UI)	2026-01-10	2026-02-15	Talha
Backend API Development (Flask/FastAPI)	2026-01-10	2026-02-15	Mubashar
Database Design & Data Structuring	2026-01-15	2026-02-10	Mubashar
Image AI Integration (Real-ESRGAN)	2026-02-20	2026-03-10	Talha
Voice AI Integration (Whisper + ElevenLabs)	2026-02-20	2026-03-15	Hassan
Text AI Integration (GPT-4 Memory Reconstruction)	2026-02-20	2026-03-20	Mubashar
Timeline Generation & Event Clustering	2026-03-10	2026-03-25	Team
Reminder & Notification System	2026-03-20	2026-04-05	Team
Frontend + Backend + AI Integration	2026-03-25	2026-04-10	Team
Testing & Bug Fixing	2026-04-05	2026-04-25	Team
Cloud Deployment (Render / Heroku)	2026-04-20	2026-04-30	Team
Final Documentation & Evaluation Preparation	2026-05-01	2026-05-10	Team

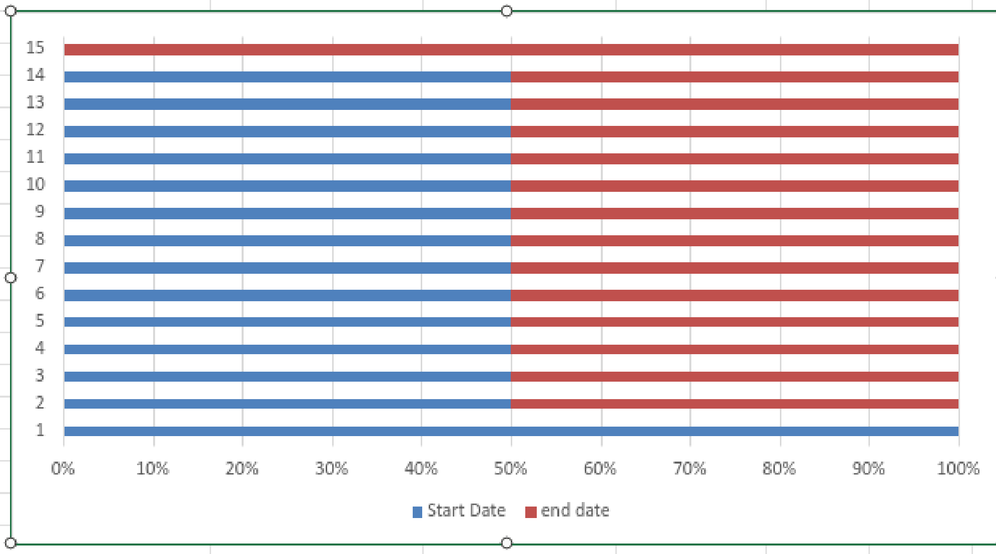


Fig 6.1: Gantt Chart

7. References

[1] Alzheimer's Disease International, *World Alzheimer Report 2022: Life After Diagnosis – Navigating Treatment, Care and Support*. London, U.K.: Alzheimer's Disease International, 2022.

[2] World Health Organization, "Dementia," WHO Fact Sheet, 2023. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/dementia

[5] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. (Draft). Stanford University, 2023. [Online]. Available: https://web.stanford.edu/~jurafsky/slp3/

[6] A. Radford *et al.*, "Whisper: Robust speech recognition via large-scale weak supervision," OpenAI, 2022. [Online]. Available: https://openai.com/research/whisper

[7] X. Wang, L. Xie, C. Dong, and Y. Shan, "Real-ESRGAN: Training real-world blind super-resolution with pure synthetic data," *arXiv preprint arXiv:2107.10833*, 2021.

[8] OpenAI, "GPT-4 Technical Report," OpenAI Research, 2023. [Online]. Available: https://openai.com/research/gpt-4

[10] ElevenLabs, "AI Voice Generation and Text-to-Speech Platform," 2024. [Online]. Available: https://elevenlabs.io

[11] PostgreSQL Global Development Group, "PostgreSQL Documentation," 2024. [Online]. Available: https://www.postgresql.org/docs/

[12] Google, "Firebase Storage Documentation," 2024. [Online]. Available: https://firebase.google.com/docs/storage

[13] R. Fielding and R. Taylor, "Principled design of the modern web architecture," *ACM Transactions on Internet Technology*, vol. 2, no. 2, pp. 115–150, 2002.

[14] A. H. Smith, "Assistive technologies for people with dementia: A review," *Journal of Assistive Technologies*, vol. 14, no. 2, pp. 85–98, 2020.

[15] S. Mohsen, "Alzheimer's disease detection using deep learning and machine learning: A review," *Artificial Intelligence Review*, vol. 58, article 262, 2025.

[16] J. Lazar, J. Feng, and H. Hochheiser, *Research Methods in Human–Computer Interaction*, 2nd ed. Cambridge, U.K.: Cambridge University Press, 2017.

[17] A. Mihailidis, J. Boger, T. Craig, and J. Hoey, "The COACH prompting system to assist older adults with dementia through handwashing: An efficacy study," *BMC Geriatrics*, vol. 8, no. 28, pp. 1–18, 2008.

[18] J. Hoey, T. Plötz, D. Jackson, A. Monk, C. Pham, and P. Olivier, "Rapid specification and automated generation of prompting systems to assist people with dementia," *Pervasive and Mobile Computing*, vol. 8, no. 1, pp. 140–154, 2012.

[19] M. A. Goodwin *et al.*, "AI-based digital memory aids for supporting individuals with Alzheimer's disease," *IEEE Journal of Biomedical and Health Informatics*, vol. 23, no. 5, pp. 2146–2154, 2019.

[20] G. Demiris, B. K. Hensel, M. Skubic, and M. Rantz, "Senior residents' perceived need of and preferences for 'smart home' sensor technologies," *International Journal of Technology Assessment in Health Care*, vol. 24, no. 1, pp. 120–124, 2008.

Appendix A: Glossary

Appendix B: IV & V Report

(Independent verification & validation)
IV & V Resource

Name Signature

S#	Defect Description	Origin Stage	Status	Fix Time	
				Hours	Minutes
1					
2					
3					
…					

Table 1: List of non-trivial defects

This document has been adapted from the following:

- Previous project templates at UCP
- High-level Technical Design, Centers for Medicare & Medicaid Services. (www.cms.gov)